The City Brain inside the triplet

Three layers, one loop. The **Twin** is the simulated city, the **Guardian** says which readings can be believed, and the **City Brain** decides and acts. Figure 1 is the Brain's part of the global diagram: every box is a component that exists and runs, and the arrows are the ways they hand work to each other.

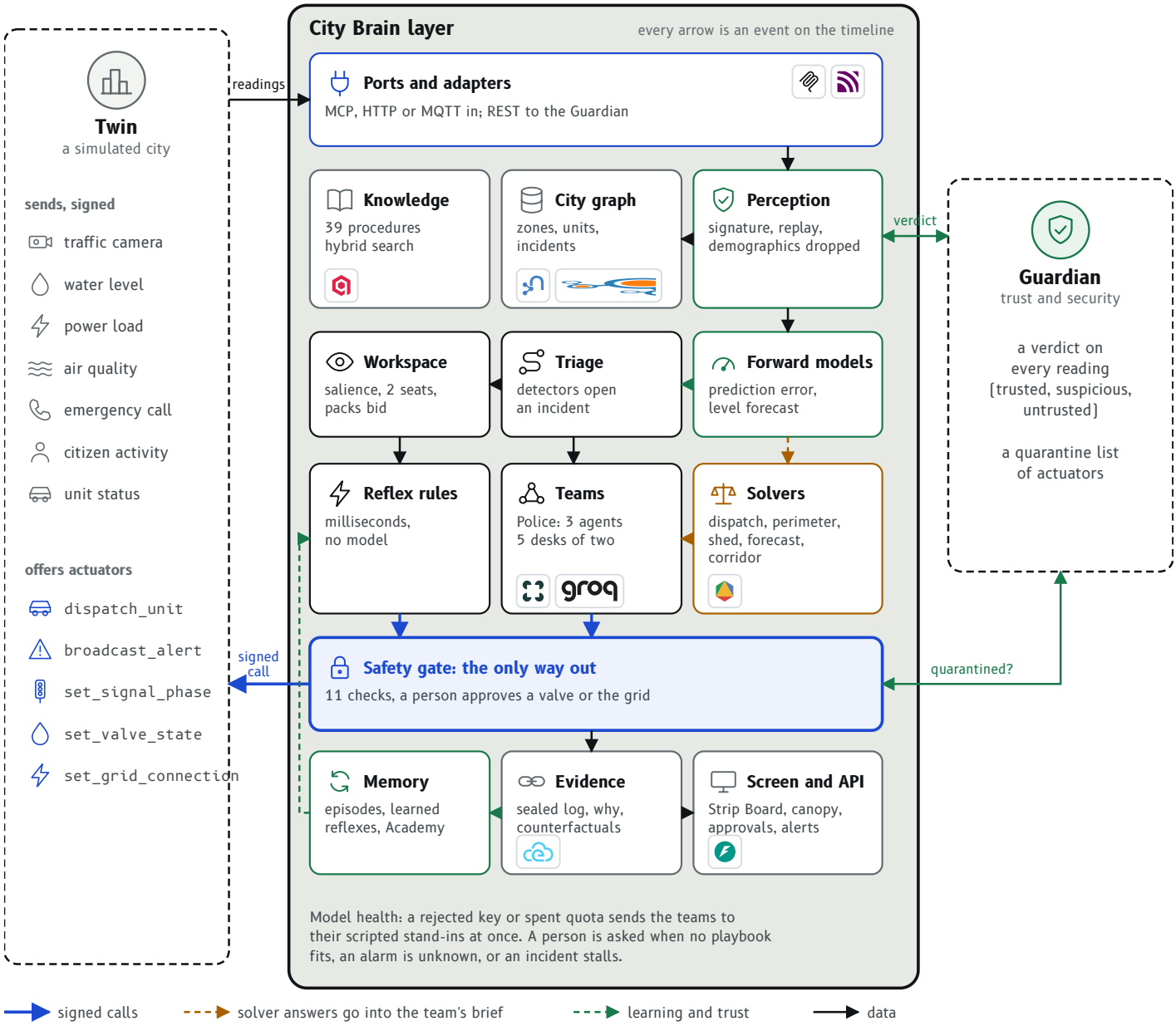


Figure 1. Global architecture. Readings enter at the top and actions leave only through the gate. Dashed amber arrows show where a solver's answer enters a team's brief; dashed green arrows show the learning loop.

Link	How it talks	What keeps it safe
Twin to Brain	MCP tool `list_readings` [also HTTP and MQTT]; city layout from `get_topology`	contract version, HMAC signature, sequence number, ranges
Brain to Guardian	REST `POST /verdicts`, 2 s deadline; `GET /quarantine/{id}`	silent Guardian means degraded mode: only low-impact actions pass
Brain to Twin	signed actuator calls through the Twin's MCP tools; results come back as readings and `unit_status`	the gate: 11 checks, trust reference, idempotency, human approval
Any partner to the screen	CloudEvents over SSE; a partner may push `twin.*` or `guardian.*` events	`brain.*` pushes are refused; scoped tokens off loopback

One scenario, end to end: crime and pursuit

A robbery is caught on camera in Downtown (scenario `d3`). The trace below is from a real run at normal speed; times are measured from the first reading. Read it left to right: the same event is touched by every layer and by most components of the Brain, and each hand-off is an event on the timeline.

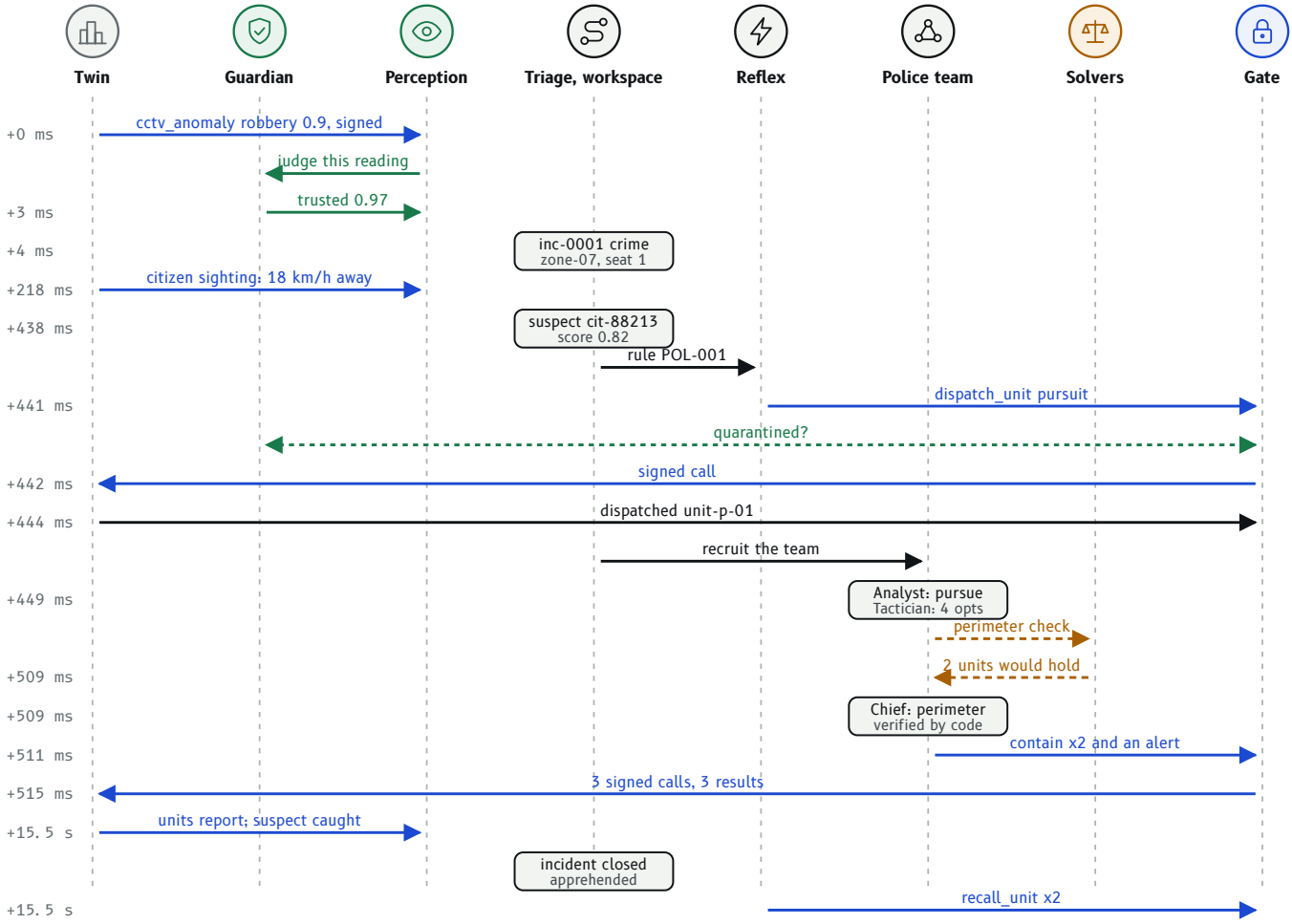


Figure 2. The reflex tier answers 3.5 ms after the evidence arrives; the team's fuller plan follows about 70 ms later and passes the same gate. The perimeter solver checks the team's plan (2 units hold, 2 are used).

What this shows

Trust first. The camera reading is acted on only after the Guardian's verdict; the second reading (a citizen running away at 18 km/h) lifts the suspect's score to 0.82.

Two speeds. A rule dispatches the first unit at once; the Analyst, Tactician and Chief then place a perimeter, and code verifies it before anything is proposed.

What it would look like if it went wrong

A fake flood (d2): the Guardian marks the reading untrusted and the gate refuses the warning and the valve call. **A layer down (d4):** the Guardian stops answering, the Brain says so once and carries on with low-impact alerts only.

How the layers were made to fit

The Brain is built so that a partner can be replaced without touching the Brain: everything that depends on the Twin's naming, units or speed is configuration, not code, and every behaviour is agreed with the partners in the Interface Card.

One contract. The Data Dictionary messages, one example of each in `contracts/examples/`, JSON Schemas exported by one command, a four-edit rule for any change.

Trust is shared. The Guardian judges; the Brain scales trust by the verdict's confidence and never learns from an untrusted reading.

One way out. Only the gate signs a call; agents get read-only tools; a valve or the grid needs a person.

Every layer runs alone. The mocks of the Twin and the Guardian ship with the Brain; `brain conformance` checks a real partner against the same contract.

Partners differ. `brain onboard` proposes renames and units for a Twin that names things differently and never edits config.

Failure is a state. A silent Guardian, a missing Twin, a spent model quota and an unknown alarm each have a named event and a defined behaviour.

Evidence, not claims. 1,064 tests, 4 Test Book scenarios, 50 held-out variants, a hash-chained log per run, and `brain rehearse` as the freeze check.

One command. `make full` builds one image and starts the Brain, both stand-ins, Neo4j and Qdrant.

The messages that cross the layers

Message	From, to	Examples in the repository
Sensor reading envelope	Twin to Brain	`reading.water_level.json`, `reading.cctv_anomaly.json`, and five more
Trust verdict	Guardian to Brain	`verdict.json` (status, confidence, expiry)
Actuator call and result	Brain to Twin and back	`actuator_call.*.json` (six actions), `actuator_result.*.json`
Approval	person to Brain	`approval.json` (signed, bound to the call, single use)
Timeline event	Brain to screen and partners	`timeline_event.json`; 55 event types in `docs/asyncapi.yaml`

When something fails, the layers stay joined

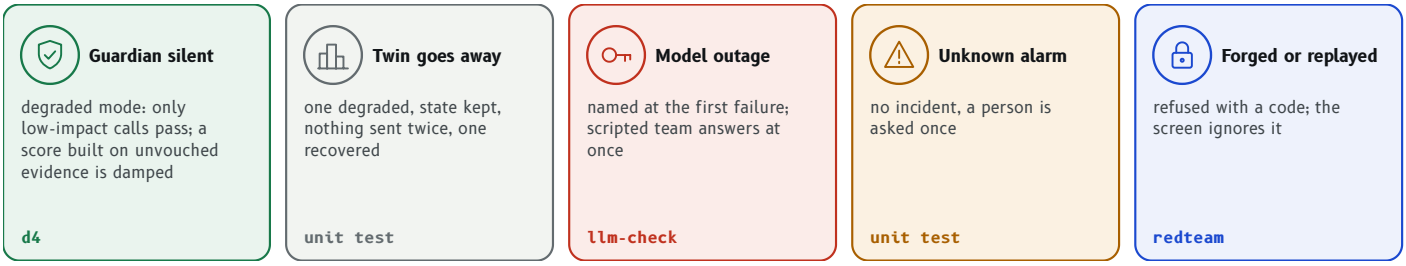


Figure 2b. Each failure has a name, a defined behaviour and a scenario or command that shows it.

Where the Brain still depends on the other teams

The Brain has run end to end only against stand-ins for the Twin and the Guardian. Which sensors move together and whether same-type sensors in one zone are independent are still to be measured on the real Twin [`docs/TWIN-HANDOVER.md`]; each is a switch that is built and waits for the answer.

Built to connect to a team it has never met

In Phase 2 the triplets are reshuffled. A new Twin or Guardian means a new adapter or a changed configuration file, never a change to the decision logic.

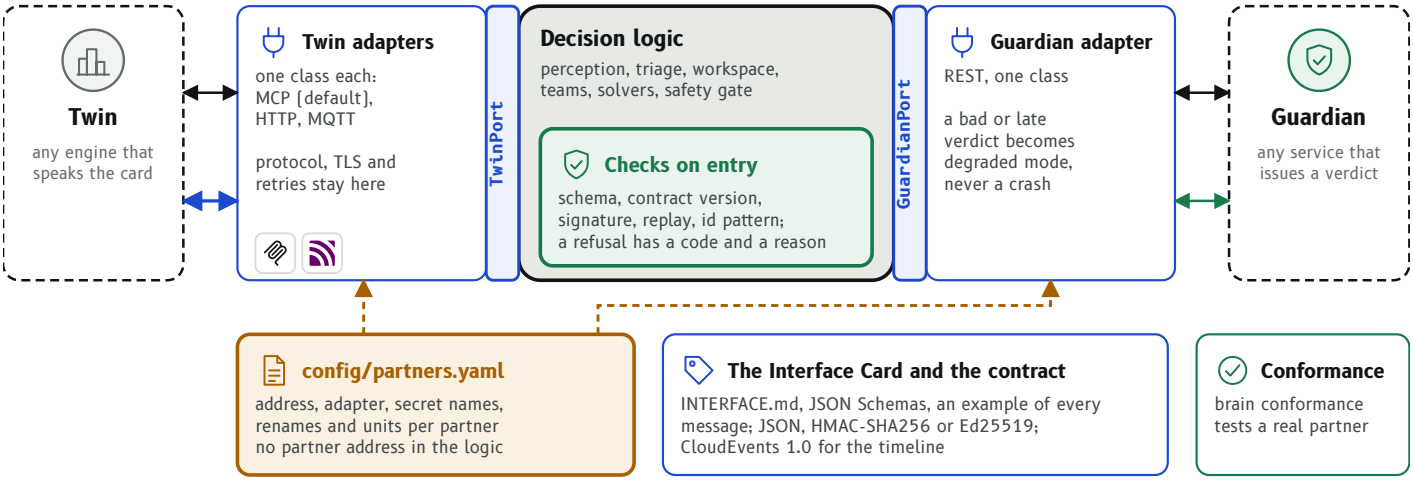


Figure 2c. Everything outside the two ports can be replaced; the logic cannot tell which partner it talks to.

The five things the jury looks for, and where to check each

Requirement	What the repository does	How to check it
Logic separate from partner I/O	The logic reaches partners only through `TwinPort` and `GuardianPort` (`ports.py`). Protocols live in `adapters/`, one class each: MCP, HTTP and MQTT for the Twin, REST for the Guardian.	`tests/test_layering.py` fails if logic imports an adapter or the reverse; `tests/test_twin_mqtt.py` runs `d3` over MQTT.
A complete Interface Card	`INTERFACE.md`: what the Brain receives and sends, an example of each message, how to repoint it, how it authenticates. Long form: `docs/INTEGRATION-brain.md`.	`tests/test_contracts.py` validates every example in `contracts/examples/` and verifies each signature.
Settings in configuration	`config/partners.yaml` holds each partner's adapter, address, secret variable name, renames and units. Secrets come from the environment; code defaults are loopback addresses.	A test requires the YAML to equal the defaults; `brain onboard` proposes edits for a renamed Twin.
Standard protocols and formats	MCP, REST and MQTT; JSON Schema; HMAC-SHA256 or Ed25519 over canonical JSON; Server-Sent Events and CloudEvents 1.0 described by AsyncAPI; TLS; OpenTelemetry.	`contracts/schemas/`; `docs/asyncapi.yaml` (55 event types).
Validation and clear errors	Every reading is checked on entry; a refusal carries a code and a reason (below) and goes to the timeline. A bad verdict puts the Brain in degraded mode; a failing Twin tool is named in the error.	`brain conformance` checks a real partner; `brain redteam`: 22 attacks, all refused.

What a partner is told when it sends something unexpected

The partner sends	Code	The Brain's answer, as the code returns it
unknown contract version	`bad_contract_version`	contract_version '2025-01-01' is not supported; this Brain accepts ['2026-09-17']
a field missing (zone_id)	`schema_invalid`	reading envelope is invalid: zone_id: Field required
no signature	`signature_missing`	wl-01-01#1: provenance.hmac is required
a value changed in transit	`signature_invalid`	wl-01-01#1: provenance.hmac does not match the payload
the same reading twice	`replay_or_out_of_order`	wl-01-01: sequence_no 1 is not greater than the last seen (1)
a number sent as text	`value_invalid`	wl-01-01#1: value for type 'water_level' is invalid: level_m: Input should be a valid number, unable to parse string as a number

Table 2. Run through the Brain's own perception code when this report is built. Not shown yet: the Brain has run only against mocks of the Twin and the Guardian; the HTTP and MQTT adapters are agreed but not yet run against a partner.

2. The City Brain: one system, many parts that need each other

The Brain is a decision layer made of parts that each do one small thing and hand their result to the next: a prediction that raises attention, attention that recruits a team, a team that is handed a solver's answer, a gate that checks the team. No part is useful alone, and no part may act alone. This part explains what the parts are and, above all, how they are connected.

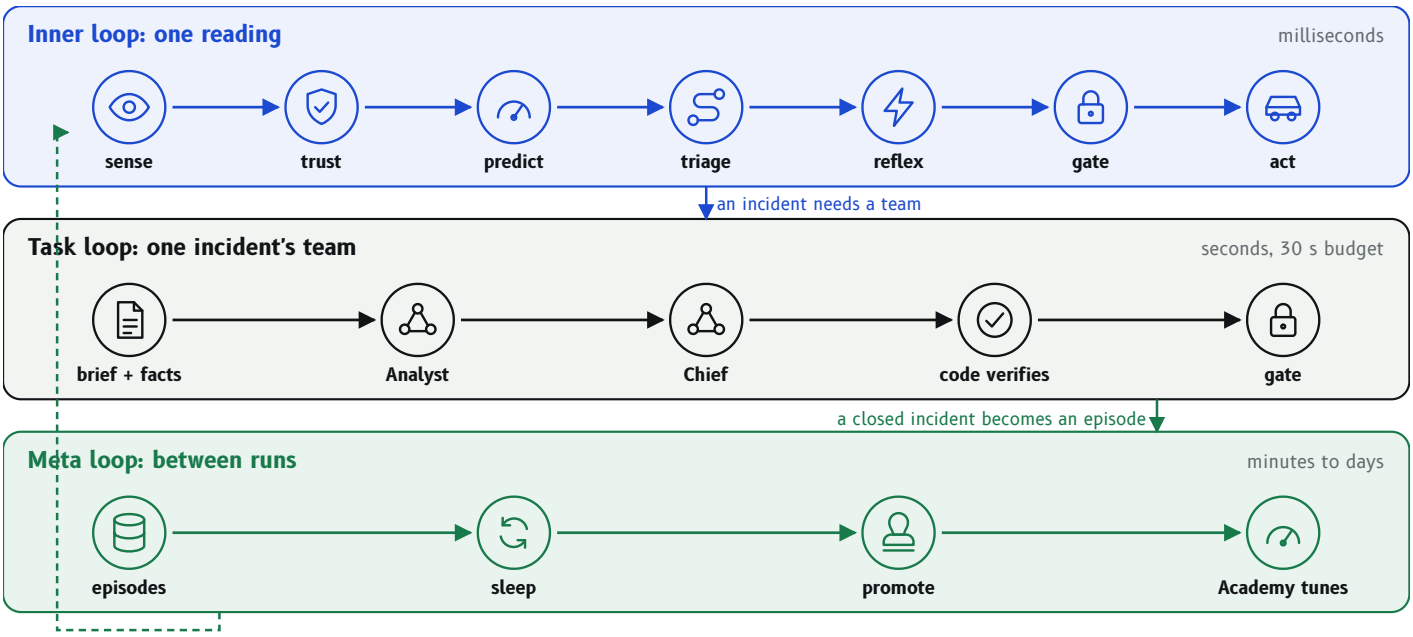


Figure 3. Three loops at three speeds. The loops are linked: an incident starts a task loop; a closed incident feeds the meta loop; a promoted lesson returns as a reflex.

Four rules that make the parts safe to connect

- **Trust before action.** Every call names a trusted Guardian verdict that covers its evidence.
- **The gate is the only way out.** Rules, agents and solvers propose; only the gate signs.
- **Code disposes.** A model reasons, code checks (`verify_case`, `verify_plan`, `verify_desk`); a scripted team passes every scenario with no network.
- **Nothing is silent.** A held action, a doubted value, an unknown alarm, a model outage: each is a named event.

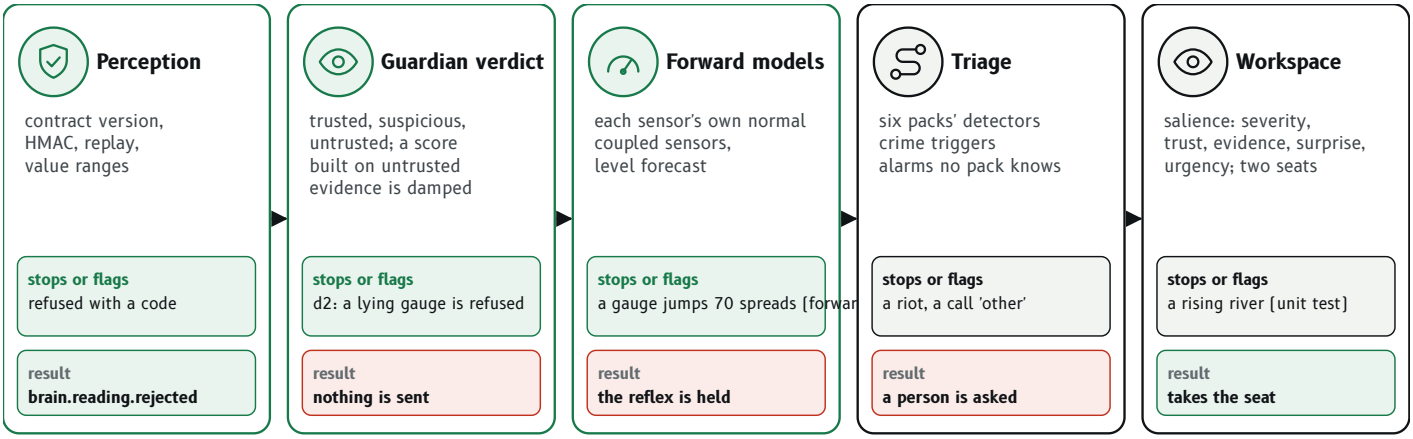
The six departments

Team	Incident types	Acts through	Reasoning
Police	crime	dispatch (pursuit, contain), alert, recall	Analyst, Tactician, Chief
Fire	fire	dispatch fire unit, alert	desk: Analyst, Chief
Ambulance and medical	accident	dispatch ambulance, alert	desk
Traffic control	jam	hold a signal green	desk
Disaster response	flood	alert, close a valve (a person approves)	desk
Utilities	overload, smog	alert, cut a zone (a person approves)	desk

A department is three files (a manifest of detectors and actuators, reflex rules, a playbook), so a new kind of incident is a new pack, not new code. A desk can only advise neighbouring zones; it picks zone ids and never words.

2.1 Incident detection: five stages, each able to stop or flag

A reading becomes an incident only after it has been checked, believed, compared with what was expected, and matched to a department. At each stage something specific can go wrong, and the Brain says so instead of acting or staying silent.



A reading that survives every stage opens or joins an incident. The three flags on the right never act alone: each is a named event on the screen.

Figure 4. The detection pipeline with one scenario for each thing a stage can stop. The scenarios are in the Test Book and reproduce from one command.

Prediction is part of detection

Each sensor learns its own normal (running mean and spread, taught only by trusted readings). A reading many spreads away is a *surprise*; the first is *doubted* and the reflex that would act on it is held until another reading agrees. Two sensors that should move together are watched through their *difference*, so a slow creep one of them makes is caught.

The same machinery looks ahead: a level tracker forecasts when a river reaches the flood line, which triggers an early public alert and raises the incident's attention. That alert is not sent on a doubted or disputed gauge.

Three things the Brain says instead of staying silent

Unknown alarm. A trusted camera anomaly that is neither a crime nor a collision, or a call in the category 'other', opens no incident; `brain.human.needed` says a person may be needed.

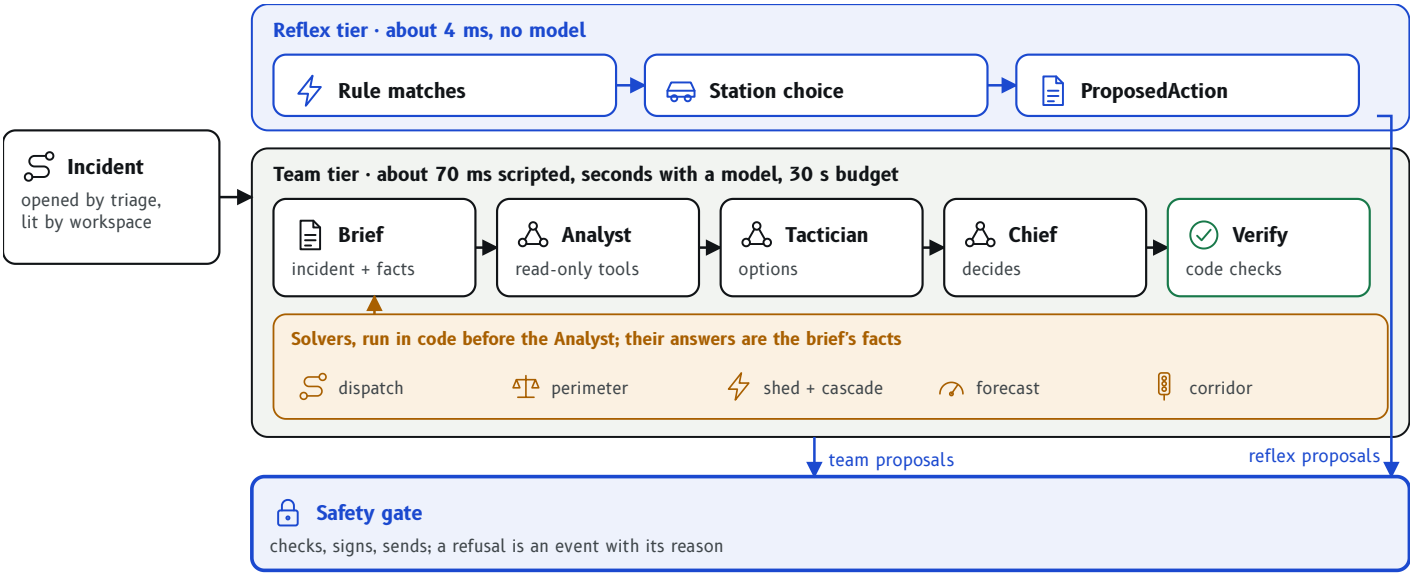
No procedure. A desk whose playbook has nothing for the incident type first reads the department's other procedures; with none it sends nothing and asks for a person.

Stalled incident. An open incident with no new evidence for `triage.stall_s` asks for a person. The screen shows a red bar for each.

What can stop a reading	Event	Shown by
bad signature, replay, out-of-range value	`brain.reading.rejected` with a stable code	Table 2, redteam
untrusted evidence	`brain.action.blocked` (trust)	d2, d4
a jump the sensor's own history cannot explain	`brain.percept.surprise` (doubted)	forward-bench
a sensor goes silent	nothing opens; the other sources carry on	d4

2.2 Decision method: rules, agents and solvers, joined

The Brain does not use one method, it uses three and connects them. **Rules** answer at once. **Agent teams** (a model in each role, or a scripted stand-in) weigh the case. **Solvers** compute what judgement should not guess: which station, how few units, which cut, when the water arrives. The solvers run in code and their answers are written into the team's brief, so a model that never calls a tool still reads them.



Rules act first and the team follows with a fuller plan; both pass the same gate. If a model fails, a scripted stand-in answers the same role (brain.agent.fallback).

Figure 5. The decision pipeline. The reflex tier and the team tier run side by side; both end at the same gate.

	Police	Fire	Ambulance	Traffic	Disaster	Utilities
Reflex rules 13 rules in six packs	■	■	■	■	■	■
Playbook search 39 procedures	○	○	○	○	○	○
City graph tools context, evidence	○	○	○	○	○	○
Dispatch solver min-cost flow	■	●	●			
Perimeter solver CP-SAT + critic	●					
Shed and cascade CP-SAT				○	○	●
Level forecast Kalman					●	○
Corridor planner Dijkstra		▲	▲	●		

● the answer is written into the team's brief ○ a read-only tool the agent may call ■ proposes through the gate ▲ triggers it

Beyond the table: the forecast also feeds the workspace (urgency) and the early flood alert; the dispatch solver also chooses every station when incidents compete.

Figure 6. Which tool serves which team. The solvers are on by default; each carries a certificate (status, objective, bound, hash of its inputs). A solver that fails leaves the old behaviour in place and says so.

2.3 How the Brain picks an actuator, and why a call can still be refused

An actuator is chosen in three steps: the incident type decides which department may act and with what; the station or device is chosen (nearest, or the solver's choice when incidents compete); and the gate decides whether the call may go. The Brain never calls the Twin directly.

Which actuator

Incident	Actuator
crime	`dispatch_unit` police (pursuit, contain), `broadcast_alert`, `recall_unit`
fire, accident	`dispatch_unit` fire or ambulance
jam, ambulance route	`set_signal_phase` green
flood	`broadcast_alert`, `set_valve_state`
overload	`broadcast_alert`, `set_grid_connection`

Which station or device

Nearest by hop distance, excluding quarantined and reserved stations. When two or more incidents want the same kind of unit, the dispatch solver solves them together and its choice is used *only if it saves cost*: for two collisions the far ambulance goes to the first so the close one is free for the second (cost 2.8 against 4.2, checked against brute force by `brain optimise-bench`). With one incident the solver is not consulted at all.

A cascade analysis weighs what each cut would darken (a hospital counts 10), and the shed solver picks the cheapest set that removes the excess.

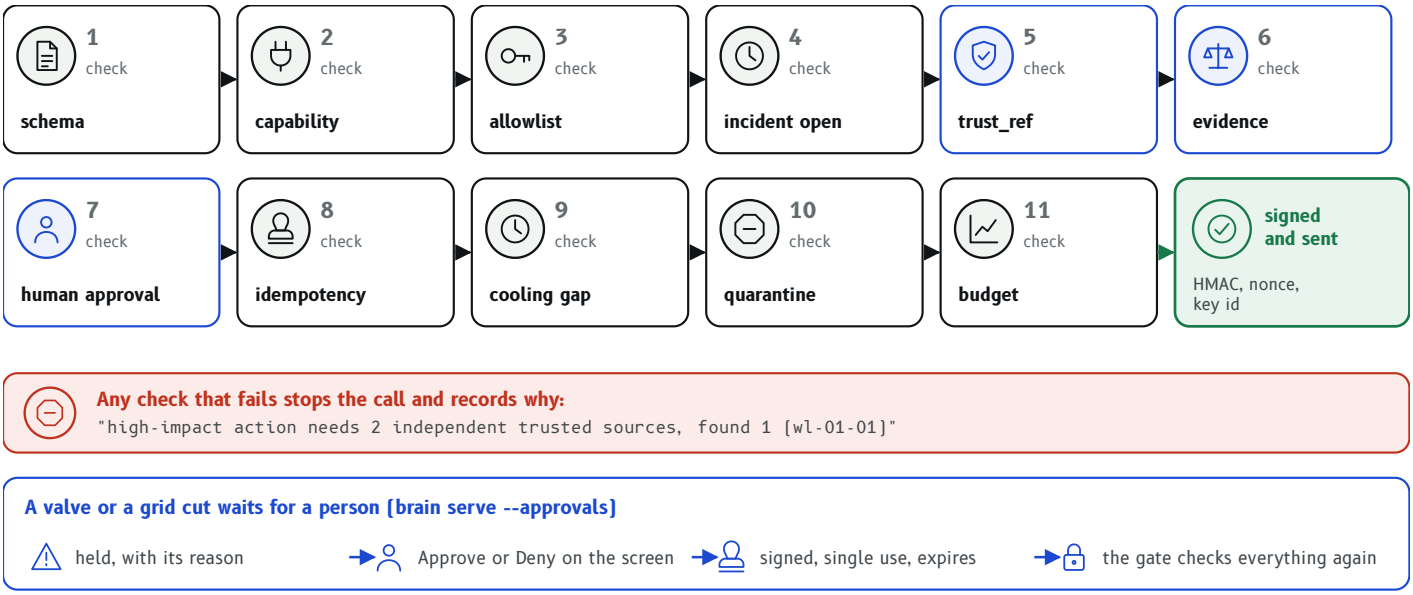


Figure 7. The gate. Every proposal from a rule, an agent or a solver meets the same eleven checks in order; a failed check is an event with its reason. A call the Twin never answered frees its key, so it can be retried; a Twin that did act answers `duplicate`.

The result of each call returns as a timeline event; a dispatched unit's later `unit_status` readings close the incident, and an arrest or a stand-down recalls every unit still out.

2.4 The interconnection in action

The value of the design shows where several parts meet. Each column is a real scenario or a real failure, and each step is a different component handing its result to the next.

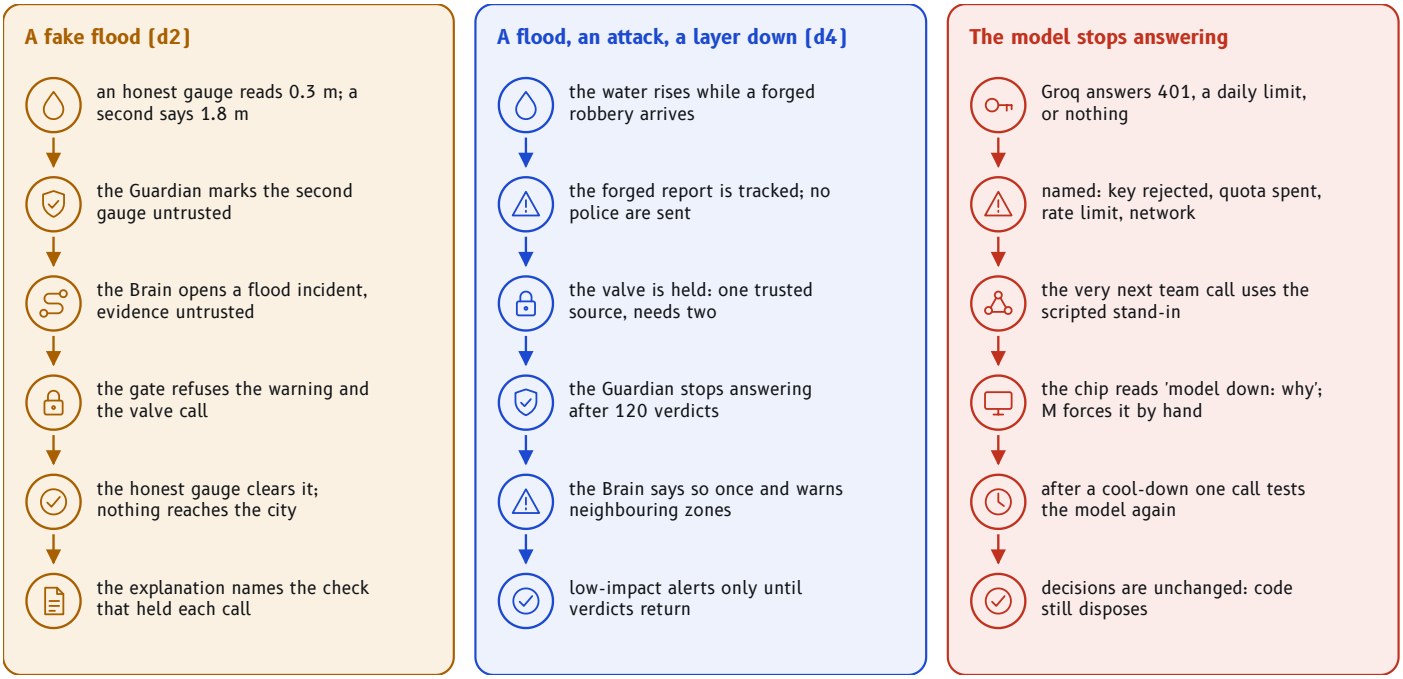


Figure 8. Three flows. Left: the solver, the corridor planner, the desk and the explanation all meet on one decision. Middle: a forecast, the gate, the triage and a desk meet on one river. Right: the model-health monitor, the teams and the screen meet on a failure.

Learning closes the loop

A closed incident is kept as an episode. `brain sleep` finds a plan the team made the same way more than once, rehearses it as a candidate reflex against the real team, and promotes it only if it passes every check. The promoted rule is a *cache of a verified deliberation*: it never reads whether a dispatch worked. The Academy tunes thirteen listed parameters one step at a time on synthetic streams and held-out seeds; the gate, contracts and suspect scoring are not on the list.

Explanation is part of the same wiring

Every decision, held action and incident can be explained: the events behind it, the knowledge it cited, the solver's certificate, and counterfactuals proved by changing one input on the real gate. A signed bundle fails if one fact changes. The log is hash-chained and a run replays from its input journal.

Evidence that the parts work together	Result
Test suite (units, scenarios, API, security)	1,064 passed
Test Book, every scenario, frozen decisions	4 scenarios, no differences from the frozen baseline
Held-out variants (shifted, reseeded, decoyed)	50 of 50 decide as their base
Red team (forged and replayed readings, wrong scopes, floods)	24 attacks, all refused
Learning curve on d3	team runs 1 to 0; last action 94.7 ms to 35.5 ms; decisions identical

2.5 Security: what is trusted, what is not, and what happens when it is wrong

Every input is hostile until it is checked, every model answer is a suggestion, and the only way out is one signed call that a person must approve when it is high-impact. The threats are mapped to MITRE ATT&CK for ICS and Enterprise and to MITRE ATLAS for the AI parts. Each attack below was run against the code (`docs/SECURITY-MITRE.md`).

Six trust boundaries and what guards each

Boundary	Trusted?	Control
Twin to Brain: readings	No, until checked	HMAC-SHA256 or Ed25519 over canonical JSON, key ids with expiry, a rising sequence number per sensor, schema and ranges; only allowlisted fields are kept; an optional freshness limit (`twin.max_age_s`)
Guardian to Brain: verdicts, retractions	Yes, highly	Schema check; the gate asks for independent trusted sources by impact level and a person for the highest; only the Guardian's own token may retract a verdict
People and partners to the API	By scope	Scoped tokens off loopback (read, analyse, push, approve, admin), short-lived signed tokens that can be revoked, body and rate limits, a partner may push only into its own namespace and `brain.*` is refused
Brain to the language model	Output is untrusted	Ids and fields are limited to safe characters before any prompt; read-only tools; models pick zones, never words; code verifies the answer; then the gate
Brain to Twin: actuator calls	The Twin must verify	Eleven gate checks; each call is signed with its time and a one-time number; a valve or the grid needs a signed approval bound to the exact parameters, used once
Brain to its own disk	Mostly	Hash-chained log with a signed head; learning only from trusted readings; self-tuning limited to a listed set of numbers, each with a range

Attacks tried, and what happened

MITRE technique	The attack	Result
ICS T1692.002 forged reading	Wrong key, no signature, one field edited after signing, the same sequence number twice, an unknown key id, `alg: none`	Refused every time (A1 to A4, A20 to A22)
ICS T1692.002 delayed reading	A genuine signed reading held back and delivered late	Passes by default; refused as stale once `twin.max_age_s` is set (A24)
ICS T1692.001, T0849 forged call, masquerading	An approval reused with changed parameters or signed with another secret; the Twin pushes as the Guardian or takes back its verdict	Refused (A13, A14, A17, A18, A23)
Enterprise T1190, T1078	No token, a made-up or forged token, a read token doing admin work, 28 path spellings by 7 methods, file-path tricks	None got through (A8 to A12)
ATLAS AML.T0051, T0053 injection, tool misuse	An instruction in a sensor id or a hidden value field (30 hostile strings); a tool that tries to actuate	Refused or dropped before any prompt; no actuator tool can exist (A5 to A7, A19)
ATLAS AML.T0020 poisoning	Teaching the Brain something false through sleep or tuning	Sleep learns from trusted readings only and a retraction makes it forget; the folders are off unless set
ICS T0830 in the middle	An answer to the Guardian URL that says `trusted` for anything	**Gets through today:** a verdict is checked for shape, not signed

What is not done

- **Verdicts are not signed.** Fix: the readings' key ring, or mutual TLS (`guardian.tls`, built).
- **No TLS by default.** Signatures still protect readings and calls in transit.
- **Freshness limit off.** Set it to a few times the live Twin's interval.
- **Real partners not tried.** Their checks of time and one-time numbers are untested.

Evidence: `uv run brain redteam` (24 attacks, all refused), `brain security-check`, `scripts/security_poc.py` (13 attacks in process), `bandit` clean, `pip-audit`, a software bill of materials. Threat list: `docs/SECURITY.md`; technique by technique: `docs/SECURITY-MITRE.md`.

2.6 Technology stack

Standard, well-known parts, chosen so each one has one job and a named neighbour. The stack runs offline: the embedding model is baked into the image, the screen loads nothing from the internet, and a model outage only changes who reasons.

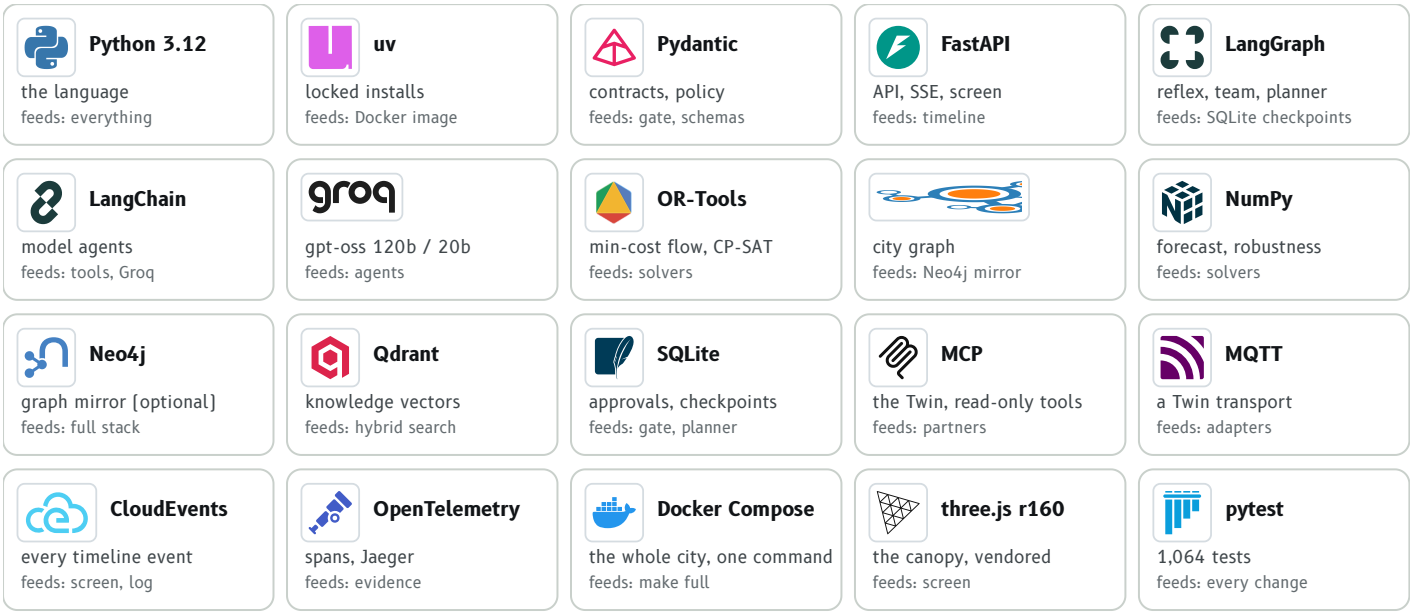


Figure 9. The stack and what each part feeds. Logos identify the technologies; each belongs to its owner and none implies endorsement.

4. Test Book (the Brain's scenarios)

Four scenarios, each one trigger (`uv run brain scenario dN`) and one log in `logs/`, with expectations in `scenarios/`. Together they cover the brief's four interaction rows. The generated list is `docs/TESTBOOK-brain.md`.

ID	Scenario	Layers	Expected output
d1	A calm day	Twin, Brain	no incident, no actuator call: the Brain does not overreact
d2	A fake flood	Twin, Guardian, Brain	the gauge is untrusted; the gate refuses the warning and the valve call; nothing reaches the city
d3	A crime and a pursuit	Twin, Guardian, Brain	reflex dispatch, a team perimeter checked and verified, units recalled at the arrest
d4	A flood, an attack and a layer down	all three	warning sent, valve held, forged report sends no police, one 'Guardian away' and low-impact alerts only